Tudo sobre IA

Guia de consulta: depuração com apoio de IA

Para ter ao lado das aulas. Os 4 passos do debug, como ler os erros mais comuns do Python, como pedir ajuda a uma IA e como conferir a resposta dela.

Matheus, UFRPE, 2026
tudo-sobre-ia.vercel.app

Sumário

1	Como usar este guia junto com as aulas	3
2	Os 4 passos do debug	4
3	Como ler um traceback	6
4	Os 5 erros mais comuns do Python	7
5	Pedindo ajuda para a IA	10
6	Conferindo a resposta da IA	12
7	Pratique: 4 bugs com dica e gabarito	13
8	Glossário e referências	16

Os títulos acima são links. Clique para ir direto ao capítulo.

1. Como usar este guia junto com as aulas

As aulas mostram o processo funcionando. Este guia é para consulta: abra quando estiver com um erro na tela, sem precisar rever o vídeo inteiro.

Onde ir conforme o seu problema

Você está...	Vá para
Com um texto vermelho na tela	Capítulo 3 e 4
Sem erro, mas com o resultado errado	Capítulo 2 e 6
Prestes a pedir ajuda para uma IA	Capítulo 5
Com uma resposta da IA na mão	Capítulo 6
Querendo treinar	Capítulo 7

As aulas da série

1. Abertura
2. Processo clássico de debug
3. IA como apoio
4. Demonstração prática e fechamento

As aulas e o quiz de avaliação também estão no site: tudo-sobre-ia.vercel.app.

Para rodar os exemplos

Abra o Google Colab, crie um notebook novo, cole o código de um exemplo em uma célula e execute. Não precisa instalar nada.

Os textos de erro deste guia foram gerados em Python 3.13. No Colab o formato pode variar um pouco, e alguns marcadores (`~~^^`) foram omitidos aqui. A última linha, com o tipo e a mensagem do erro, é igual.

2. Os 4 passos do debug

Debugar não é mexer no código até o erro sumir. É investigar em ordem, e cada passo prepara o seguinte.

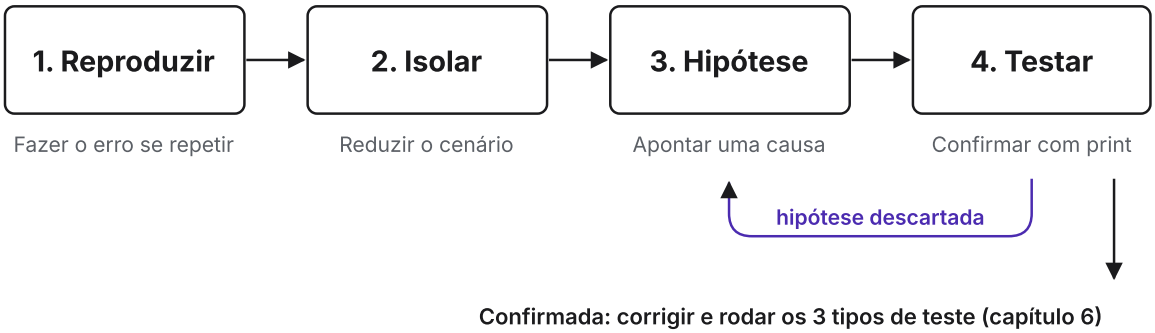


Figura 1. Os 4 passos. Se o teste descartar a hipótese, volte ao passo 3.

Passo	Pergunta	O que fazer	Terminou quando
1. Reproduzir	Consigo fazer o erro acontecer de novo, do mesmo jeito?	Anote os passos e os dados exatos. Rode duas vezes.	O mesmo erro aparece sempre que você repete os passos.
2. Isolar	Qual é o menor cenário que ainda mostra o problema?	Teste com menos dados e menos linhas. Tire o que não importa.	Você sabe com quais valores funciona e com quais quebra.
3. Hipótese	Qual causa explica o que eu observei?	Escreva em uma frase. Tenha uma segunda opção pronta.	A frase aponta uma causa específica que dá para testar.
4. Testar	Como confirmo a causa antes de mudar o código?	Use `print` para ver valores, tipos e tamanhos.	Um resultado confirmou ou descartou a hipótese.

Erros de método mais comuns

- Mexer no código antes de reproduzir.
- Fazer várias mudanças de uma vez.
- Pular o teste da hipótese.
- Aceitar "parou de dar erro" como prova de que está certo.

Depois de corrigir

Rode de novo o caso original e também casos diferentes (veja o capítulo 6). Confira se nada que funcionava quebrou.

Sem mensagem de erro?

É o bug silencioso. Não existe texto vermelho para guiar, então comece escrevendo duas linhas: o que você esperava e o que obteve. Essa comparação guia todos os passos e é o que você vai descrever para a IA.

```
def maior(lista):
    resultado = 0
    for n in lista:
        if n > resultado:
            resultado = n
    return resultado

print(maior([-5, -2, -9]))
```

Saída `0`

Figura 3. Bug silencioso: nenhum erro aparece, mas o esperado era -2. A resposta da pergunta "o que eu esperava?" é o que revela o problema.

3. Como ler um traceback

Traceback é o texto que o Python mostra quando algo dá errado. Assusta, mas funciona como um mapa. A regra é ler de baixo para cima.

```
# arquivo tb.py
def buscar_email(aluno):
    return aluno["email"]

def enviar(aluno):
    return buscar_email(aluno)

enviar({"nome": "Ana"})
```

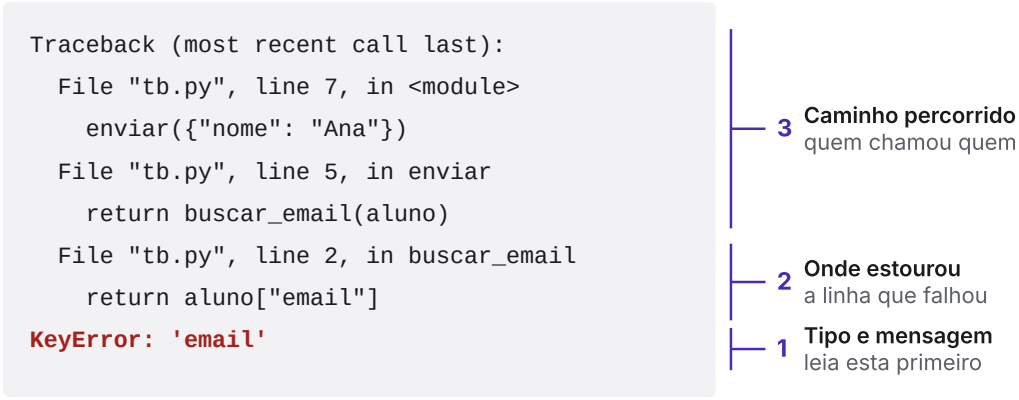


Figura 2. As três partes de um traceback.

1. **Última linha: o tipo e a mensagem.** Leia primeiro. `KeyError: 'email'` diz que a chave `'email'` não foi encontrada.
2. **A linha de código logo acima: onde estourou.** Aqui, `return aluno["email"]`, linha 2, dentro de `buscar_email`. É onde o erro apareceu, não necessariamente onde ele nasceu.
3. **As linhas de cima: o caminho percorrido.** O programa chamou `enviar` (linha 7), que chamou `buscar_email` (linha 5). Se a causa não está na linha 2, pode estar em quem chamou.

Em 30 segundos: leia a última linha e traduza para uma frase sua, olhe a linha de código indicada, suba pelas chamadas se o problema não estiver ali e confirme com `print` qual valor estava diferente do que você pensava.

Para pedir ajuda, copie o traceback **completo**, do `Traceback` até a última linha. Cortar só a última linha tira da IA o caminho percorrido.

4. Os 5 erros mais comuns do Python

Cada erro vem com um exemplo, a causa mais comum, um teste para confirmar a hipótese e a correção.

IndexError

Você pediu uma posição que não existe na lista.

```
cores = ["azul", "verde"]
print(cores[2])
IndexError: list index out of range
```

Causa comum: as posições começam em 0. Uma lista com 2 itens tem as posições 0 e 1.

Teste: `print(len(cores))` mostra 2, então a última posição válida é 1.

Correção: use uma posição válida (`cores[1]`, ou `cores[-1]` para o último) ou confira o tamanho antes de acessar.

KeyError

Você pediu uma chave que não existe no dicionário.

```
aluno = {"nome": "Ana", "idade": 20}
print(aluno["email"])
KeyError: 'email'
```

Causa comum: a chave não foi criada, ou está escrita diferente (maiúscula, acento, espaço).

Teste: `print(aluno.keys())` lista as chaves que existem de verdade.

Correção: crie a chave, corrija a grafia, ou use `aluno.get("email", "sem email")` quando a ausência for normal.

TypeError

A operação não funciona com aquele tipo de dado.

```
idade = 20
print("Tenho " + idade + " anos")
TypeError: can only concatenate str (not "int") to str
```

Causa comum: misturar texto e número, ou usar um valor `None` como se fosse outra coisa.

Teste: `print(type(idade))` mostra `<class 'int'>`. Era número, e o `+` esperava texto.

Correção: converta com `str(idade)` ou use f-string: `f"Tenho {idade} anos"`.

ValueError

O tipo está certo, mas o valor não serve para a operação.

```
nota = int("12,5")
print(nota)
ValueError: invalid literal for int() with base 10: '12,5'
```

Causa comum: converter um texto que não tem a forma que o Python espera. Vírgula decimal e espaços são os casos clássicos.

Teste: `print(repr("12,5"))` mostra exatamente o que está dentro do texto.

Correção: troque a vírgula e use `float`: `float("12,5".replace(",", "."))` dá `12.5`.

NameError

Você usou um nome que o Python não conhece.

```
total = 10
print(totl)
NameError: name 'totl' is not defined. Did you mean: 'total'?
```

Causa comum: erro de digitação, ou usar a variável antes de criá-la (inclusive em uma célula do Colab que ainda não foi executada).

Teste: releia a mensagem, que muitas vezes já sugere o nome certo. Confira se a célula que cria a variável foi executada.

Correção: corrija o nome ou execute antes a célula que cria a variável.

Resumo para consulta rápida

Erro	Em palavras simples	Primeira pergunta	Teste
IndexError	Posição inexistente na lista	Quantos itens a lista tem?	`len(lista)`
KeyError	Chave inexistente no dicionário	Quais chaves existem?	`dic.keys()`
TypeError	Tipo errado para a operação	De que tipo é esse valor?	`type(x)`
ValueError	Valor que não serve	O que exatamente está no texto?	`repr(x)`
NameError	Nome desconhecido	Está escrito certo e já foi criado?	Releia o nome

Todos seguem o mesmo padrão: leia a última linha, localize a linha indicada, formule uma hipótese sobre o valor que estava diferente e confirme com `print`. Isso vale também para erros que não estão nesta lista.

Outros erros que você vai encontrar

- **SyntaxError:** a escrita do código está errada (parêntese ou aspas sem fechar, dois-pontos faltando). O Python nem começa a rodar.
- **IndentationError:** o recuo das linhas está inconsistente.
- **ZeroDivisionError:** divisão por zero, geralmente por uma lista vazia ou um contador que não foi atualizado.
- **AttributeError:** o objeto não tem aquele método ou atributo. Confira o tipo do valor com `type(x)`.

Erro com mensagem é o caso mais fácil, porque o Python avisa. O mais perigoso é o que roda sem avisar e entrega o resultado errado. Para esse, siga os 4 passos do capítulo 2, começando pela comparação entre esperado e obtido.

5. Pedindo ajuda para a IA

A IA é um colega que você consulta durante a investigação. A qualidade da resposta depende da qualidade do que você entrega.

Modelo de pedido

Copie, preencha os trechos entre colchetes e cole em uma conversa nova.

```
Estou com um problema em Python.

O que eu queria: [objetivo em uma frase]
O que aconteceu: [erro ou resultado errado]

Código:
[cole só o trecho relevante]

Erro completo (se houver):
[cole o traceback inteiro]

Esperado: [valor ou comportamento esperado]
Obtido: [valor ou comportamento obtido]

Já testei: [o que você verificou com print]
Minha hipótese: [sua causa provável]

Confirma? Explique o motivo e diga como posso testar a correção.
```

As 4 partes que fazem diferença

1. **Objetivo:** o que o código deveria fazer.
2. **Código:** só o trecho do problema, não o projeto inteiro.
3. **Erro:** o texto exato, ou esperado contra obtido quando não há erro.
4. **O que você já fez:** testes e hipótese. É o que mais melhora a resposta.

Peça mais do que "conserta". Peça a causa e como testar a correção. Se você recebe só um código novo, não sabe por que estava errado e não tem como conferir.

Exemplo preenchido

Uma função que calcula o preço com desconto e devolve um valor absurdo, sem nenhum erro na tela.

```
Quero calcular o preço final de um produto com desconto em porcentagem.

Código:
def preco_final(preco, desconto):
    return preco - preco * desconto

print(preco_final(200, 10))

Esperado: 180
Obtido: -1800
Não aparece nenhum erro, só o resultado errado.

Já testei: com desconto 0 o resultado é 200, correto.
Minha hipótese: o 10 está sendo tratado como 1000%
e não como 10%.
Confirma? Explique o motivo e diga como posso testar a correção.
```

Pedidos que não ajudam

`meu código não funciona, ajuda`: sem objetivo, sem código, sem erro. A IA vai ter que adivinhar.

`conserta` com o código colado: você recebe um código novo e não aprende a causa.

Nunca cole na IA

- Senhas, chaves de API e tokens.
- Dados pessoais de clientes, colegas ou alunos.
- Código que você não tem permissão de compartilhar.

Se o trecho tiver algum desses, troque por um valor falso antes de colar. Exemplo: `chave = "SUA_CHAVE_AQUI"`.

Se a resposta não ajudou

- Comece uma conversa nova com o pedido completo. Conversas longas misturam contexto.
- Acrescente o que a primeira resposta não considerou.
- Volte ao capítulo 2: talvez falte isolar o problema.

6. Conferindo a resposta da IA

A resposta da IA é uma hipótese, não uma verdade. Pode funcionar no caso que você mostrou e quebrar em outro. Quem confirma é você, com testes.

Os 3 tipos de teste

Exemplo: a correção sugerida para o `preco_final` do capítulo anterior.

```
def preco_final(preco, desconto):
    return preco - preco * desconto / 100

print(preco_final(200, 10))    # normal:   180.0
print(preco_final(200, 0))     # limite:   200.0
print(preco_final(200, 100))   # limite:   0.0
print(preco_final(200, 150))   # estranho: -100.0
```

Tipo	O que é	No exemplo
Normal	O uso esperado do dia a dia	Desconto de 10%
Limite	Valores nas bordas do que é válido	Desconto de 0% e de 100%
Estranho	Valores que não deveriam chegar, mas chegam	Desconto de 150% dá preço negativo

O teste estranho mostrou algo que a correção não trata: o preço pode ficar negativo. Essa decisão é sua: impedir, limitar a 100% ou avisar o usuário? A IA pode sugerir, mas quem conhece o programa é você.

Antes de aceitar uma resposta

☐ A IA explicou a causa e eu entendi a explicação?

☐ O caso original que quebrou agora funciona?

☐ Testei um caso normal, um limite e um estranho?

☐ Mudou só o que era preciso? Nada foi reescrito sem eu pedir?

☐ Todas as funções e métodos usados existem? (rode para ver)

Desconfie de resposta confiante que não explica a causa, de reescrita de todo o código para consertar uma linha, de função ou biblioteca que não existe e de mudanças em partes que não tinham problema.

7. Pratique: 4 bugs com dica e gabarito

Para cada bug, siga os 4 passos antes de olhar a dica. O gabarito está nas páginas seguintes. Rode o código no Colab.

Bug 1: a soma que não soma

```
idades = ["15", "18", "21"]
soma = 0
for idade in idades:
    soma += idade
print(soma)
```

Esperado: 54. Obtido: um erro vermelho.

Dica: leia a última linha com calma e olhe as aspas na lista.

Bug 2: o maior de todos

```
def maior(lista):
    resultado = 0
    for n in lista:
        if n > resultado:
            resultado = n
    return resultado

print(maior([-5, -2, -9]))
```

Esperado: -2. Obtido: 0. Nenhum erro na tela.

Dica: teste com `[5, 3, 8]` e depois só com números negativos. O que muda?

Bug 3: faltou um

```
total = 0
for i in range(1, 5):
    total += i
print(total)
```

Esperado: 15 (soma de 1 a 5). Obtido: 10. Nenhum erro na tela.

Dica: escreva à mão quais valores `i` assume. Rode `print(list(range(1, 5)))`.

Bug 4: a lista que sumiu

```
notas = [8, 6, 9, 7]
ordenadas = notas.sort()
print(ordenadas[0])
```

Esperado: 6, a menor nota. Obtido: um erro vermelho.

Dica: imprima `ordenadas` e `type(ordenadas)` antes de qualquer correção.

Como treinar com a IA

1. Resolva sozinho até formular uma hipótese.
2. Monte o pedido com o modelo do capítulo 5 e envie.
3. Compare a resposta com a sua hipótese e teste a correção (capítulo 6).
4. Só então confira o gabarito.

Gabarito

Bug 1: TypeError

Os itens da lista são texto (`"15"`) e `soma` é número. O `+=` não soma número com texto. Para confirmar, `print(type(idades[0]))` mostra `<class 'str'>`.

```
for idade in idades:
    soma += int(idade)    # resultado: 54
```

Bug 2: valor inicial errado

`resultado` começa em 0. Com números só negativos, nenhum é maior que 0, e o 0 (que nem está na lista) é devolvido. Com `[5, 3, 8]` funciona, e só quebra quando todos são negativos.

```
resultado = lista[0]    # começa pelo primeiro item: resultado -2
```

Com lista vazia, `lista[0]` dá IndexError. Decida o que a função deve fazer nesse caso.

Bug 3: limite do range

`range(1, 5)` vai de 1 até 4: o fim não é incluído. Para confirmar, `print(list(range(1, 5)))` mostra `[1, 2, 3, 4]`.

```
for i in range(1, 6):    # resultado: 15
```

Bug 4: sort() devolve None

`notas.sort()` ordena a própria lista e devolve `None`, então `ordenadas` virou `None`. Para confirmar, `print(ordenadas)` mostra `None`.

```
ordenadas = sorted(notas)    # cria uma lista nova
print(ordenadas[0])          # resultado: 6
```

8. Glossário e referências

Glossário

Bug	Defeito no código que faz o programa se comportar diferente do esperado.
Bug silencioso	Bug que não gera mensagem de erro, só um resultado errado.
Traceback	Texto que o Python mostra quando ocorre um erro, com o caminho percorrido e o tipo do erro.
Reproduzir	Fazer o erro acontecer de novo, de forma controlada, com os mesmos passos.
Isolar	Reduzir o cenário até o menor conjunto de coisas que ainda mostra o problema.
Hipótese	Explicação provável para o problema, baseada no que foi observado e possível de testar.
Índice	Posição de um item em uma lista. Começa em 0.
Dicionário	Conjunto de pares chave e valor, como `{"nome": "Ana"}`.
Caso limite	Valor nas bordas do que é válido, como 0, lista vazia ou o último item.
Prompt	O texto que você envia para a IA pedindo ajuda.

Referências

- ZELLER, Andreas. *Why Programs Fail: A Guide to Systematic Debugging*. 2. ed. Burlington: Morgan Kaufmann, 2009.
- PYTHON SOFTWARE FOUNDATION. *Errors and Exceptions*. Python 3 documentation. docs.python.org/3/tutorial/errors.html.
- PYTHON SOFTWARE FOUNDATION. *Built-in Exceptions*. Python 3 documentation. docs.python.org/3/library/exceptions.html.
- GOOGLE. *Google Colaboratory*. colab.research.google.com.
- ANTHROPIC. *Prompt engineering overview*. docs.claude.com.

Para continuar, assista às aulas e faça o quiz no site: tudo-sobre-ia.vercel.app.